kmhelpers: A Python toolkit for automated management of genomic indexes

Sébastien Bellenous[1] **and Pierre Peterlongo**[1]

1 Genscale, Univ. Rennes, Inria, CNRS, IRISA - UMR 6074, Rennes, F-35000 France

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Large-scale genomic sequence search has become a central problem in modern bioinformatics (Karasikov et al., 2025; Marchet, 2024). A widely used family of methods relies on Bloom filters (BF) (Bloom, 1970) to record, for each indexed sample, which k-mers (words of length k) it contains; a query then reports the set of samples containing each queried k-mer. Building such an index over many samples is not a single operation but a chain of interdependent steps, and each step demands specialist knowledge: sizing each Bloom filter to its sample, configuring the third-party components used internally by `kmindex` (such as the `findere` (Robidou & Peterlongo, 2021) approximate-membership layer), distributing data and computation, bounding peak RAM, and grouping samples into sub-indexes so that the final index stays small without slowing queries. An error at any step can invalidate downstream results, waste significant computation, or produce an oversized index.

We present `kmhelpers`, an open-source Python toolkit that automates this entire workflow for indexes built with `kmindex` (Lemane et al., 2024). It hides the technical decisions listed above behind a command-line interface (CLI) and a matching Python API that cover every stage of the k-mer index lifecycle, from raw samples to queries, including the federation of independently built indexes into a single queryable resource.

Statement of Need

k-mer-based sequence databases built with tools such as `kmindex` (Lemane et al., 2024) and `kmtricks` (Lemane et al., 2022) answer large-scale questions in genomics, metagenomics, and population studies; they were recently used to index 50 petabases of raw sequencing data (Chikhi et al., 2024). Yet a wide gap separates having the indexing software installed from running a reproducible, end-to-end, size-optimised indexing workflow.

Crossing that gap currently requires a user to master, simultaneously: Bloom filter sizing as a function of each sample's k-mer cardinality; the configuration of `kmindex`'s internal components; the distribution of data and build jobs across the available compute; the control of peak memory; and the partitioning of samples into sub-indexes that balances index size against query speed. Only specialists hold all of this knowledge at once.

`kmhelpers` removes that barrier. It makes the complete process of building, updating, and querying a `kmindex` search engine accessible to any group that holds a large, evolving genomic dataset — research teams, sequencing platforms, hospitals, or data-production centres — whether the resulting index is kept private or shared. This opens to non-specialists both the creation and maintenance of private indexes and the contribution of sub-indexes to larger, federated search engines.

Formally, the input is a set $\mathcal{S} = \{S_1, \dots, S_n\}$ of n genomic samples of various sizes; each S_i is either a raw sequencing dataset or assembled sequences. The output is a `kmindex` index

The Journal of Open Source Software

subject to two user-defined limits: (1) the maximum allowed false-positive rate, and (2) the maximum number of sub-indexes, each a file storing a set of BFs as a matrix. `kmhelpers` orchestrates every step between input and output, automating the parametrisation and the data distribution.

State of the Field

Several tools tackle large-scale sequence search with k-mer-based data structures. BIGSI (Bradley et al., 2019) and COBS (Bingmann et al., 2019) use compressed Bloom filter matrices to answer presence/absence queries across collections of sequencing experiments. HowDeSBT (Harris & Medvedev, 2020) and Mantis (Pandey et al., 2018) rely on sequence Bloom trees and counting quotient filters, respectively. More recent colored-k-mer indexes such as MetaGraph (Karasikov et al., 2025) and Fulgor (Fan et al., 2024) push scalability further; see (Marchet, 2024) for a survey. `kmindex`, which `kmhelpers` wraps, builds on the `kmtricks` (Lemane et al., 2022) counting pipeline and is designed for efficient querying of large sequence collections.

`kmhelpers` does not compete with any of these approaches at the algorithmic level. Its contribution is orthogonal and, importantly, is not merely a pipeline script: the difficulty here lies not in chaining steps but in the individual components and how they are orchestrated. `kmhelpers` implements the decisions a `kmindex` expert would otherwise make by hand: `profile` computes Bloom filter sizes and sample-to-sub-index assignments that minimise total index size under a false-positive-rate constraint; `plan` estimates disk and memory requirements before any build starts; and `registry` federates independently built sub-indexes into one logical index. No dedicated automation layer of this kind existed for `kmindex` prior to `kmhelpers`.

Design and Implementation

Background: the sub-index structure

A Bloom-filter index stores all BFs of the same size as a single (row-major) matrix in one file. In such a matrix each column is one BF (one sample) and each row records the presence (1) or absence (0) of a k-mer across all samples sharing that filter size, assuming a common hash function. A complete index is therefore a set of such matrices, each one a *sub-index* holding all BFs of a given size.

The difficulty is that each BF size must match the number of items it stores — here, the distinct k-mers of a sample. If all samples had the same number of distinct k-mers, every BF would share one size and a single matrix would suffice: the ideal case, where one file is opened per query and one matrix row gives the answer across all n samples. In practice, sample sizes differ by several orders of magnitude. Sizing every BF for the largest sample wastes enormous storage; giving each sample its own single-column matrix minimises storage but forces a query to open n files (potentially millions). `kmhelpers` takes the middle ground: the user fixes the maximum number of sub-indexes, and the `profile` step chooses the per-sub-index BF size that minimises total index size under that constraint, and it distributes input samples to their respective sub-index.

The pipeline

`kmhelpers` exposes the index lifecycle as a sequence of commands, illustrated Figure 1:

- **`list`** — recursively discovers all samples in a given directory and counts each sample's distinct k-mers using `ntCard` (Mohamadi et al., 2017) (unless the counts are provided by the user).
- **`profile`** — determines the best set of sub-index BF sizes given the user-defined maximum number of sub-indexes and target false-positive rate.

The Journal of Open Source Software

- **`compose`** — assigns each sample to its sub-index and generates the *files-of-files* describing the data origin of each sub-index.
- **`plan`** — validates sample files, available disk space, and memory upfront, and emits ready-to-execute pipeline scripts.
- **`apply`** — builds all sub-indexes by invoking `kmindex`, with span-level and name-level filtering.
- **`compress`** — optional ZSTD-based block compression of the index (**?**).
- **`registry`** — registers several sub-indexes (built locally or anywhere accessible) into one logical index, redirecting each query to the relevant sub-indexes at query time.

Once an index is built, `kmhelpers` also answers queries (`query`). Multi-step workflows can be described as declarative YAML pipelines (`pipeline`) and executed in a single command.

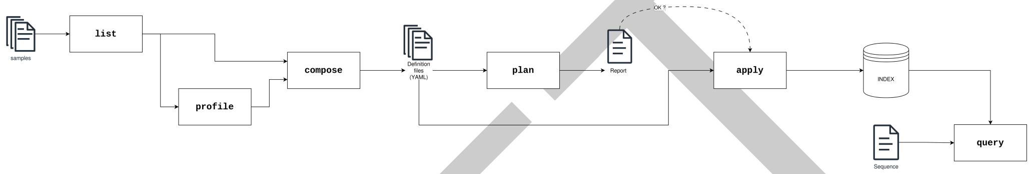

Figure 1: Overview of the `kmhelpers` workflow. `list` enumerates sample files and counts k-mers; `profile` analyses the count distributions to assign each sample to a Bloom-filter span and recommend index groupings. Both outputs feed into `compose`, which generates YAML index definition files. `plan` then validates sample paths, available disk space and memory, and emits a ready-to-execute pipeline script; the resulting report is reviewed before committing to the build. `apply` reads the definition files and invokes `kmindex` to construct the index. Finally, `query` searches the built index against user-provided sequences and returns ranked results. TODO: remove redundant info already in the main text

Implementation

`kmhelpers` is implemented in Python (≥ 3.8) and distributed via Conda, which installs its bioinformatics dependencies (`kmindex`, `ntCard`) automatically. The CLI is built with Click (Ronacher & Click Contributors, 2023), and the package exposes a public Python API covering all CLI functionality. Together, the declarative YAML index-definition format and the `plan`/`apply` separation make k-mer indexing workflows accessible to researchers who are not experts in the underlying data structures, while remaining flexible enough for large-scale production use.

Tree of Life demonstration

TODO real-world demonstration pending experimental results. Required content when results arrive: dataset description (number of samples, total size), hardware used, index parameters chosen automatically by kmhelpers, output index size, build wall time, and query throughput.

Availability and documentation

`kmhelpers` is released under the GNU General Public License v3.0 and is available at https://github.com/sebllns/kmhelpers. The full documentation is available at https://sebllns.github.io/kmhelpers/.

Acknowledgements

The authors thank Téo Lemane for developing `kmindex` and for his responsiveness in addressing feature requests and issues raised during the development of `kmhelpers`. We acknowledge the

GenOuest core facility (https://www.genouest.org) for providing the computing infrastructure. The work was funded by the Inria Challenge "OmicFinder" (https://project.inria.fr/omicfinder/), and by the state funding managed by the French National Research Agency under the France 2030 program [ANR-22-PEAE-0005].

AI Usage Disclosure

AI assistance was used for constrained tasks (drafting, editing, code suggestions) under strict human review at every stage. AI provider used: Claude (Anthropic, 2025).

References

Bingmann, T., Bradley, P., Gauger, F., & Iqbal, Z. (2019). COBS: A compact bit-sliced signature index. *String Processing and Information Retrieval (SPIRE 2019)*. https://doi.org/10.1007/978-3-030-32686-9_21

Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, *13*(7), 422–426. https://doi.org/10.1145/362686.362692

Bradley, P., Bakker, H. C. den, Rocha, E. P. C., McVean, G., & Iqbal, Z. (2019). Ultrafast search of all deposited bacterial and viral genomic data. *Nature Biotechnology*, *37*(2), 152–159. https://doi.org/10.1038/s41587-018-0010-1

Chikhi, R., Lemane, T., Loll-Krippleber, R., Montoliu-Nerin, M., Raffestin, B., Camargo, A. P., Miller, C. J., Fiamenghi, M. B., Agustinho, D. P., Majidian, S., Autric, G., Hugues, M., Lee, J., Faure, R., Curry, K. D., Moura de Sousa, J. A., Rocha, E. P. C., Koslicki, D., Medvedev, P., … Babaian, A. (2024). Logan: Planetary-scale genome assembly surveys life's diversity. *bioRxiv*. https://doi.org/10.1101/2024.07.30.605881

Fan, J., Khan, J., Singh, N. P., Pibiri, G. E., & Patro, R. (2024). Fulgor: A fast and compact k-mer index for large-scale matching and color queries. *Algorithms for Molecular Biology*, *19*, 3. https://doi.org/10.1186/s13015-024-00251-9

Harris, R. S., & Medvedev, P. (2020). Improved representation of sequence Bloom trees. *Bioinformatics*, *36*(3), 721–727. https://doi.org/10.1093/bioinformatics/btz662

Karasikov, M., Mustafa, H., Danciu, D., Kulkov, O., Zimmermann, M., Barber, C., Rätsch, G., & Kahles, A. (2025). Efficient and accurate search in petabase-scale sequence repositories. *Nature*, *647*(8091), 1036–1044.

Lemane, T., Lezzoche, N., Lecubin, J., Pelletier, E., Lescot, M., Chikhi, R., & Peterlongo, P. (2024). Indexing and real-time user-friendly queries in terabyte-sized complex genomic datasets with kmindex and ORA. *Nature Computational Science*, *4*(2), 104–109. https://doi.org/10.1038/s43588-024-00596-6

Lemane, T., Medvedev, P., Chikhi, R., & Peterlongo, P. (2022). Kmtricks: Efficient and flexible construction of Bloom filters for large sequencing data collections. *Bioinformatics Advances*, *2*(1), vbac029. https://doi.org/10.1093/bioadv/vbac029

Marchet, C. (2024). Advances in colored k-mer sets: Essentials for the curious. *arXiv Preprint arXiv:2409.05214*. https://doi.org/10.48550/arXiv.2409.05214

Mohamadi, H., Khan, H., & Birol, I. (2017). ntCard: A streaming algorithm for cardinality estimation in genomics data. *Bioinformatics*, *33*(9), 1324–1330. https://doi.org/10.1093/bioinformatics/btw832

Pandey, P., Almodaresi, F., Bender, M. A., Ferdman, M., Johnson, R., & Patro, R. (2018). Mantis: A fast, small, and exact large-scale sequence-search index. *Cell Systems*, *7*(2), 201–207.e4. https://doi.org/10.1016/j.cels.2018.05.021

Robidou, L., & Peterlongo, P. (2021). Findere: Fast and precise approximate membership query. *String Processing and Information Retrieval: 28th International Symposium, SPIRE 2021, Lille, France, October 4–6, 2021, Proceedings 28*, 151–163.

Ronacher, A., & Click Contributors. (2023). *Click: Python composable command line interface toolkit*. https://click.palletsprojects.com